Algorithmique · P1 à P6

Ces programmes sont des exemples de cours. Pour les comprendre, préciser les entrées, la sortie et les hypothèses, puis exécuter quelques étapes à la main. Les réponses des activités et des DM restent à rédiger séparément.

P1 · Termes, somme et seuil

On suppose n entier positif ou nul et le plafond de sécurité entier positif. La première fonction renvoie u_n et $\sum_{k=0}^{n} u_k$ pour $u_{k+1} = au_k + b$. La seconde cherche un premier franchissement croissant ; son plafond de sécurité signale seulement qu'il n'a pas été trouvé dans le nombre d'étapes fixé.

```
def terme_somme(n, u0, a, b):
    u, total = u0, u0
    for k in range(n):
        u = a*u + b
        total += u
    return u, total

def seuil_croissant(u0, a, b, cible, maximum=100000):
    u, n = u0, 0
    while u <= cible and n < maximum:
        u = a*u + b
        n += 1
    if u <= cible:
        return None
    return n
```

P2 · Dichotomie

Les calculs ci-dessous utilisent des nombres flottants ; la garantie mathématique suppose les signes correctement déterminés. Hypothèses : f continue sur $[a, b]$ et $f(a)f(b) \leq 0$. Pour identifier un unique zéro, prouver séparément l'unicité. La fonction renvoie un intervalle de longueur au plus ε, tant que la précision machine permet encore de partager l'intervalle.

```
def dichotomie(f, a, b, epsilon):
    assert a < b and epsilon > 0
    fa, fb = f(a), f(b)
    assert fa*fb <= 0
    if fa == 0: return a, a
    if fb == 0: return b, b
    while b-a > epsilon:
        m = (a+b)/2
        if m == a or m == b:
            raise ArithmeticError("precision machine atteinte")
        fm = f(m)
```

```
        if fm == 0: return m, m
        if fa*fm < 0:
            b, fb = m, fm
        else:
            a, fa = m, fm
    return a, b
```

P3 · Euler

Avec N entier strictement positif et $t_0 < \text{fin}$, pour $y' = F(t, y)$, $Y_{k+1} = Y_k + hF(t_k, Y_k)$. Le programme renvoie toutes les approximations ; il ne prouve pas leur précision.

```
def euler(F, t0, y0, fin, N):
    assert N > 0 and fin > t0
    h = (fin-t0)/N
    y, valeurs = y0, [(t0, y0)]
    for k in range(N):
        t = t0+k*h
        y = y+h*F(t, y)
        valeurs.append((t0+(k+1)*h, y))
    return valeurs
```

P4 · Rectangles

Pour N entier strictement positif et f continue et croissante sur $[a, b]$, le résultat donne une borne inférieure et une borne supérieure de l'intégrale. Sans monotonie, ce sont seulement deux approximations à examiner.

```
def rectangles(f, a, b, N):
    assert N > 0 and a < b
    h = (b-a)/N
    gauche = h*sum(f(a+k*h) for k in range(N))
    droite = h*sum(f(a+k*h) for k in range(1, N+1))
    return gauche, droite
```

P5 · Loi binomiale et simulation

On suppose n entier positif ou nul, $0 \leq p \leq 1$ et k entier. Le calcul direct ci-dessous convient aux tailles modestes utilisées dans les exercices ; pour de très grands paramètres, des bibliothèques numériques spécialisées sont préférables.

```
from math import comb
from random import Random

def probabilite(n, p, k):
    if k < 0 or k > n: return 0.0
    return comb(n, k)*p**k*(1-p)**(n-k)

def cumul(n, p, k):
    return sum(probabilite(n, p, j)
               for j in range(min(k, n)+1))

def simuler_binomiale(n, p, generateur):
    return sum(generateur.random() < p for _ in range(n))

rng = Random(2026)
```

P6 · Fluctuations d’une fréquence

On suppose N entier strictement positif et $0 \leq p \leq 1$. La graine permet de reproduire le même exemple. Répéter avec d’autres graines ; la loi des grands nombres ne dit pas que l’écart observé diminue à chaque étape.

```
from random import Random

def frequences(N, p, graine=2026):
    rng = Random(graine)
    succes, resultat = 0, []
    for n in range(1, N+1):
        succes += rng.random() < p
        resultat.append(succes/n)
    return resultat
```